Solution: Pow(x,n)

Problem Statement

Write a Recursive Approach to Calculate Power of Integer to N Pow(x,n).

Given an integer `x` and an integer `n`, write a recursive function to calculate the power of `x` to the `n` th power.

Example

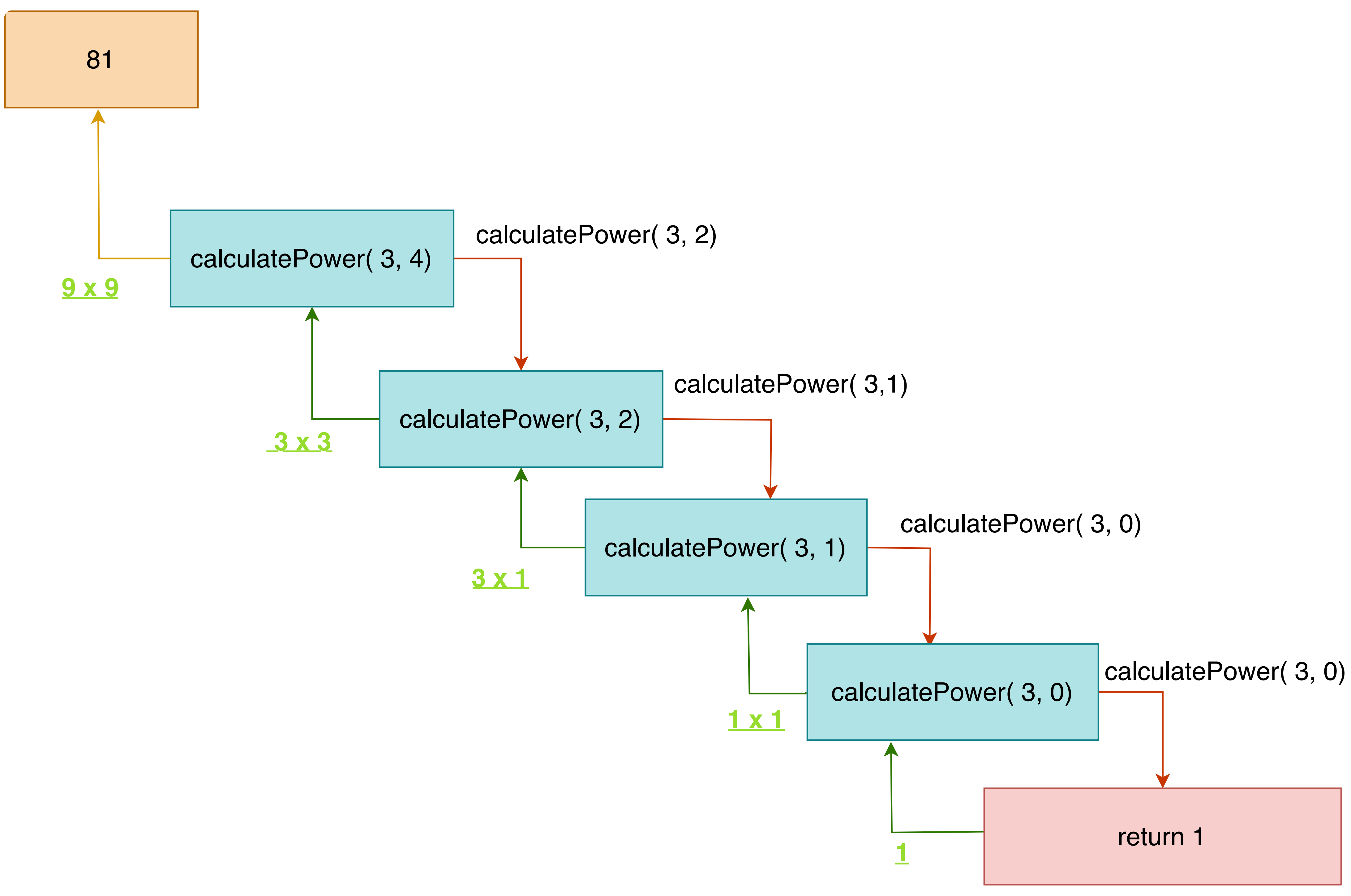
Sr#	x	n	Output	Description
1	2	5	32	2 raised to the power of 5 equals 32.
2	3	4	81	3 raised to the power of 4 equals 81.
3	5	0	1	Any number raised to the power of 0 equals 1 (x^0 = 1).

Solution

To calculate the power of `x` to the `n` th power recursively, we can follow these steps:

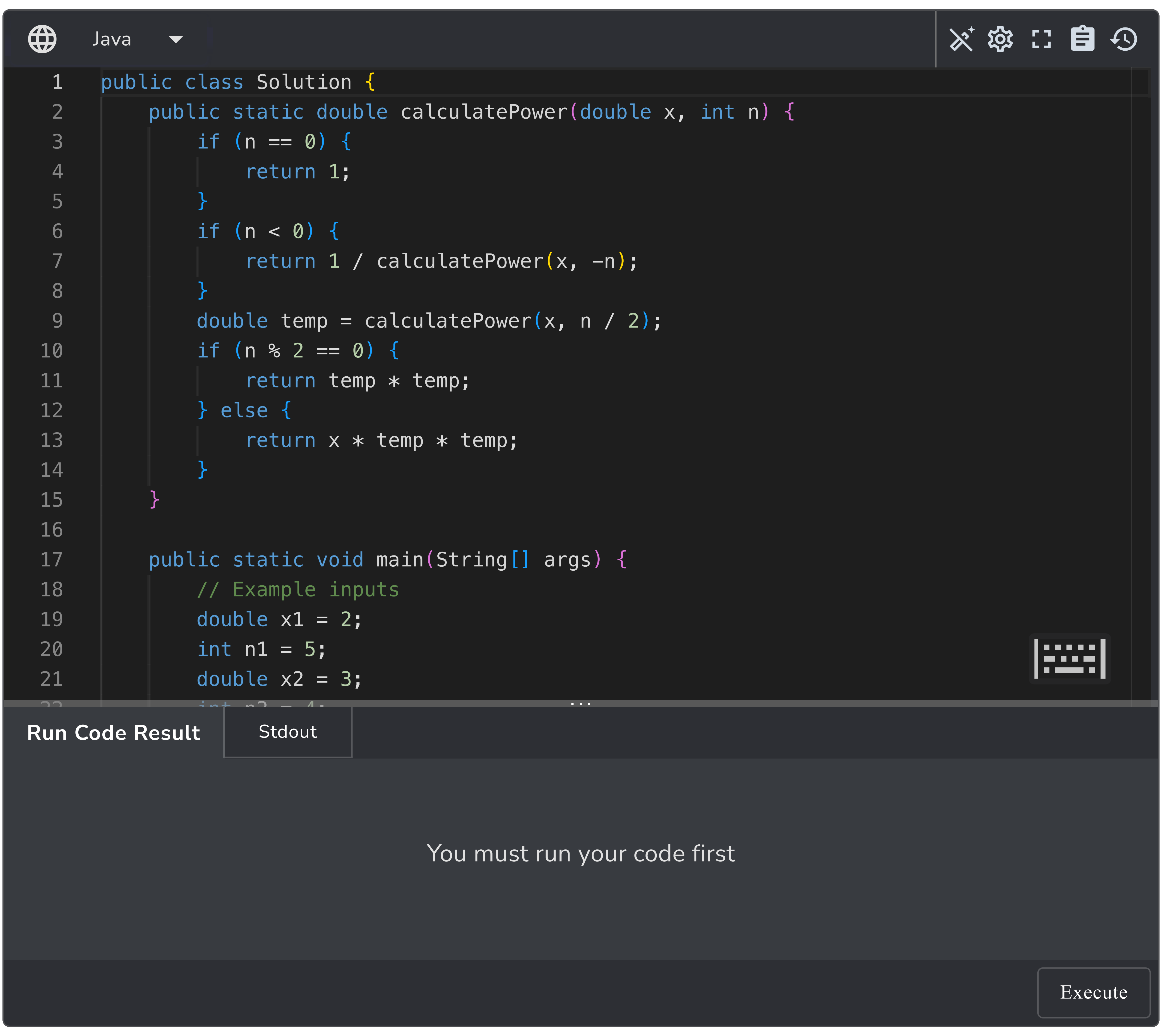
1. If `n` is equal to 0, return 1 since any number raised to the power of 0 equals 1.
2. If `n` is less than 0, return 1 divided by the result of calculating `x` raised to the power of `-n`.
3. If `n` is greater than 0, recursively calculate `x` raised to the power of `n/2`. Store the result in a variable `temp`.
4. If `n` is even, return the square of `temp` (i.e., `temp * temp`).
5. If `n` is odd, return `x` multiplied by the square of `temp` (i.e., `x * temp * temp`).

This approach works by dividing the power `n` into smaller subproblems, reducing the number of recursive calls. By utilizing the property that `x^(2k)` is equal to `(x^k)^2`, we can optimize the computation and reduce the time complexity.



Code

Here is the code for this algorithm:



```java
public class Solution {
    public static double calculatePower(double x, int n) {
        if (n == 0) {
            return 1;
        }
        if (n < 0) {
            return 1 / calculatePower(x, -n);
        }
        double temp = calculatePower(x, n / 2);
        if (n % 2 == 0) {
            return temp * temp;
        } else {
            return x * temp * temp;
        }
    }

    public static void main(String[] args) {
        // Example inputs
        double x1 = 2;
        int n1 = 5;
        double x2 = 3;
```

You must run your code first

Time and Space Complexity

The time complexity of this algorithm is $O(logN)$ because we divide the problem size in half with each recursive call. The space complexity is $O(logN)$ as well because the maximum depth of the recursion is log n.

← Previous
19. Pow(X,N)

☐ Mark as Completed